Final Report: Large-Scale Analysis of European Air Traffic

Team members (Group 14):

- Pedro González Núñez
- Nacho Castilla Llorca
- Diego Colin Reyes

---

1. What we did

We built an end-to-end big data pipeline over European ADS-B traffic. Live state vectors from the OpenSky Network are buffered in Kafka, stored in an object store, cleaned and grouped into flight trajectories with Spark, analysed (holding patterns, terminal-area congestion, traffic by time of day) and shown in a local dashboard. Run instructions: repository `README.md`.

2. Dataset

Dataset	Source	Size	Status
Live state vectors	OpenSky REST `/states/all`, Europe (lat 35–62, lon −12–30), one snapshot every 15–30 s, OAuth2 client	1,097 snapshots, **2,281,579** distinct state vectors, 10,440 aircraft	**Real**
Historical 2019, 2020, 2023	Generated by `ingest_historical.py`	137 flights, 3,568 rows	**Simulated**

The live capture ran in two blocks (29 Oct 19:34 – 30 Oct 01:47 UTC and 30 Oct 08:26 – 12:40 UTC, about 10.5 h). It is too large to ship and is regenerated by `stream_opensky_kafka_oauth.py`. OpenSky's historical access (Trino) was not granted to our account, so the 2019/2020/2023 files are synthetic (6 hub airports, injected noise and holding loops). Every row is labelled `dataset=synthetic_<year>`; they only validate the pipeline and **no real conclusion about those years is drawn**.

3. Methodology

- **Infrastructure (Docker Compose):** MinIO (buckets `airtraffic-raw`, `airtraffic-processed`), Kafka, Spark 3.5, MongoDB 7 with `2dsphere` indexes. The dashboard runs in a local virtualenv.
- **Landing (Spark, 3a):** reads the whole Kafka topic, explodes each payload into one row per aircraft state, de-duplicates on (`icao24`, time) and writes Parquet to MinIO.
- **Cleaning and sessionization (Spark, 3b):** rows with missing/invalid coordinates or speed above Mach 1 (343 m/s) are dropped. Per aircraft, a window function (`lag` plus a cumulative sum) starts a new flight session after a gap longer than 30 minutes, and consecutive repeated positions are removed. Each session of at least 2 points becomes a GeoJSON `LineString` (with per-point times, headings, speeds and altitudes) in MongoDB, with a deterministic `_id` so reruns are idempotent.
- **Holding detection (Spark, 4a):** a window over each session sums the signed heading change in a sliding 20-minute window. A holding is flagged when the sum reaches 360° inside a ~50 km box (speed ≥ 75 m/s, ≥ 6 pings, ≥ 2 min); consecutive hits are merged into one event in `holding_events`.
- **Congestion (MongoDB, 4b):** `$geoIntersects` finds the trajectories crossing each TMA polygon. Aircraft are then counted per 10-minute bucket only while their pings lie inside the area (counting whole sessions overestimated the peaks about 3×).
- **Time of day (4c):** distinct aircraft per 10-minute bucket, averaged per hour (UTC), and holdings per hour.
- **Dashboard (Phase 5):** Streamlit with Kepler.gl (or a pydeck basemap): map of trajectories, TMAs and holdings, congestion charts, time-of-day trends and a holdings table.

4. Results

4.1 Cleaning and trajectories

Metric	Value
Input rows (live + simulated)	2,285,147
Dropped: invalid coordinates / speed above Mach 1	3 / 120

Metric	Value
Clean rows	2,285,024
Trajectories (live / simulated)	23,435 (23,298 / 137)

Live sessions last 33 min on median (max 258 min); 12,099 last at least 30 min and 2,591 at least 2 h. They are observed segments, not complete flights.

4.2 Holding patterns

160 events from 127 aircraft (median duration 8.8 min). Many are military, training or surveillance orbits (for example MRL82, 2,342° of accumulated turn) next to a few commercial holds, because we do not filter by operator.

4.3 Time of day (real data, UTC)

Window (UTC)	Aircraft per 10 min (hourly average)	Holdings started
08:00 – 12:59 (Fri)	2,550 – 3,950	138
19:00 – 23:59 (Thu)	790 – 1,620	21
00:00 – 01:59 (Fri)	340 – 470	1

The late-morning peak (09:00–12:59) has about 11× the traffic of the night, and 138 of the 160 holdings (86%) start between 08:00 and 12:59. The evening block is a Thursday and the morning a Friday, and only 12 hours are covered, so this is a descriptive profile, not a tested effect.

4.4 Congestion per TMA (live data)

TMA (60 km area)	London	Paris	Frankfurt	Amsterdam	Madrid	Munich
Flights crossing it	1,994	1,597	1,136	916	910	866
Peak aircraft per 10 min	196	123	85	73	67	66

London is the busiest TMA, followed by Paris and Frankfurt; the peaks fall between 10:40 and 12:10 UTC.

5. Deviations from the proposal and limitations

Item in the proposal	What we did and why
COVID-19 comparison (2019 vs 2020 vs 2023)	**Not achieved with real data.** OpenSky historical access was not granted, so those years are simulated and only validate the batch pipeline. Instead we compare time-of-day windows on the real live data (4.3).
Diversions	**Not computed.** Live state vectors carry no planned destination and the simulated flights never divert.
Hourly arrival throughput, route frequencies	**Replaced by airborne density**: distinct aircraft inside each TMA per 10 min (not simultaneous aircraft). We have no flight lists for real traffic.
Sessions by time gaps or ground transitions	Time gap only (30 min). Live sessions are observed segments, not complete flights.
TMA polygons	Simplified 60 km circles around the main airport, not official boundaries (the GeoJSON file was not available).
Coverage	About 10.5 h of live data with gaps (no 02:47–09:26 and no 13:40–20:34, Madrid time). Day-level conclusions are limited.

6. How to run

Full instructions (prerequisites, every job with its exact command, OpenSky credentials, dashboard): `README.md`, sections 7–11. In short, with the virtualenv active:

```
docker compose up -d && python src/setup_infra.py
python src/ingest_historical.py && python src/load_tma_polygons.py
python src/stream_opensky_kafka_oauth.py --interval 30 --iterations 500
# Spark jobs (README 7-8): phase3_kafka_to_minio ->
phase3_clean_sessionize -> phase4_holdings
python src/phase4_congestion.py && python src/phase4_hourly.py
python -m streamlit run src/dashboard.py
```